AWS & DEVOPS COMPLETE STUDY NOTES

A Beginner-Friendly Guide to DevOps Fundamentals

LINUX • AWS • GIT & GITHUB • DOCKER
JENKINS • TERRAFORM • KUBERNETES • EKS

Learn -> Run -> Break -> Troubleshoot -> Explain

Notes created by Vaibhav Kunghatkar

Table of Contents

01 Linux Fundamentals
02 Linux Users, Permissions & Processes
03 Shell Scripting
04 Git & GitHub
05 AWS Services - Main Notes (EC2, S3, IAM, VPC)
06 AWS Services - Compute, Database & Traffic
07 AWS Services - Monitoring & Serverless
08 Docker
09 Jenkins & CI/CD
10 Terraform
11 Kubernetes
12 AWS EKS
13 DevSecOps & Monitoring
14 Complete DevOps Pipeline
15 Troubleshooting Checklist
16 Study Method
17 Final Revision Notes

1. Linux Fundamentals

Definition — Linux:

Linux is an open-source operating system that runs most servers, cloud instances and containers. DevOps engineers use the command line to manage files, users, processes, services, networking and applications, locally and over SSH.

Navigation & Files

```
pwd
ls -lah
cd /var/log
mkdir -p project/logs
touch file.txt
cp -r src/ backup/
mv old.txt new.txt
rm file.txt
rm -rf olddir/        # careful: no undo
ln -s /opt/app/current /opt/app/live
tree -L 2             # directory tree (may need install)
```

- **pwd** — shows the current working directory.
- **ls -lah** — long listing, hidden files, human-readable sizes.
- **mkdir -p** — creates nested directories in one go.
- **cp -r / mv / rm -rf** — copy directories, move/rename, delete recursively (double-check the path).
- **ln -s** — creates a symbolic link, handy for 'current release' folders.

Viewing, Logs & Search

```
cat app.log
less app.log                      # q to quit, /text to search
head -n 10 app.log
tail -n 50 app.log
tail -f /var/log/nginx/access.log  # follow live
grep -rin "error" /var/log/myapp/
grep -v "DEBUG" app.log
find /var/log -name "*.log" -mtime -1
find . -type f -size +100M
wc -l app.log
sort app.log | uniq -c | sort -nr | head
awk '{print $1}' access.log | sort | uniq -c | sort -nr | head
sed -i 's/old/new/g' config.txt
cut -d: -f1 /etc/passwd
```

- **grep -rin** — recursive, case-insensitive search with line numbers.
- **find -mtime -1** — files modified in the last day.
- **awk / sed / cut** — slice columns, replace text, split on delimiters.
- **sort | uniq -c** — classic pipeline to count repeated lines, e.g. top client IPs.

Practical point: *tail -f plus grep is the fastest live debugger: tail -f app.log | grep --line-buffered ERROR*

Packages, Archives & Remote Access

```
sudo apt update && sudo apt install -y nginx     # Debian / Ubuntu
sudo dnf install -y nginx                        # Amazon Linux 2023 / RHEL
tar -czvf backup.tar.gz /data
tar -xzvf backup.tar.gz -C /restore
rsync -avz ./site/ user@host:/var/www/site/
scp file.txt ubuntu@1.2.3.4:/tmp/
ssh -i key.pem ubuntu@1.2.3.4
ssh-keygen -t ed25519 -C "you@example.com"
curl -fsSL https://example.com/install.sh -o install.sh
```

- **apt / dnf** — package managers: Ubuntu/Debian use apt, Amazon Linux 2023 and RHEL family use dnf.
- **tar -czvf / -xzvf** — create / extract compressed archives.
- **rsync** — efficient incremental copy, local or over SSH.
- **ssh-keygen -t ed25519** — modern, short and secure key type; chmod 400 on private keys.

2. Linux Users, Permissions & Processes

Users & Groups

```
whoami
id
sudo -i
sudo useradd -m -s /bin/bash devuser
sudo passwd devuser
sudo groupadd devops
sudo usermod -aG devops devuser
sudo visudo                      # safely edit sudoers
last -n 5                        # recent logins
who
```

- **id** — UID, GID and group membership.
- **useradd -m -s** — creates user with home directory and login shell.
- **usermod -aG** — appends to a supplementary group (always use -a, otherwise groups are replaced!).
- **visudo** — edits sudoers with syntax checking.

Permissions

```
ls -l script.sh
chmod 755 script.sh        # rwxr-xr-x
chmod 644 file.txt         # rw-r--r--
chmod 400 key.pem          # SSH keys: owner read only
chmod +x deploy.sh
chown ubuntu:ubuntu file.txt
chown -R www-data:www-data /var/www
umask
```

Permission digits: r=4, w=2, x=1. Three digits apply to owner, group and others. Example: 755 = 7 (rwx) / 5 (r-x) / 5 (r-x).

Processes & Services (systemd)

```
ps aux | grep nginx
top            # or: htop
pgrep -a nginx
kill <PID>     # SIGTERM (graceful)
kill -9 <PID>  # SIGKILL (last resort)
systemctl status nginx
sudo systemctl start|stop|restart|reload nginx
sudo systemctl enable --now nginx
systemctl list-units --type=service --state=failed
journalctl -u nginx -f
journalctl -u nginx --since "1 hour ago" -p err
journalctl -xe
```

- **enable --now** — start at boot and start immediately.
- **--state=failed** — quickly lists services that crashed.
- **journalctl -u X -f** — follow a service's logs live.
- **kill -9** — force kill; try plain kill first so the app can clean up.

Disk, Memory & Cron

```
df -h
du -sh /var/* | sort -h
free -h
lsblk
uptime
lsof -i :8080              # who is using port 8080
crontab -e
# m h dom mon dow  command
0 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1
```

Linux Networking

```
ip addr
ip route
ss -tulpn
ping -c 4 8.8.8.8
curl -I https://example.com
curl -v http://localhost:8080/health
nslookup example.com
dig +short example.com
nc -zv db.internal 5432     # TCP port check
traceroute example.com
sudo ufw status             # Ubuntu firewall
```

- **ss -tulpn** — listening TCP/UDP ports with owning process.
- **nc -zv host port** — tests whether a port is reachable.
- **dig +short** — clean DNS answer.
- **curl -v** — full request/response detail for debugging.

3. Shell Scripting

Definition — Shell Scripting:

A shell script is a small automation program made from Linux commands. DevOps engineers use scripts for deployments, backups, health checks, log cleanup and server setup.

```
#!/bin/bash
set -euo pipefail          # stop on error, undefined vars, pipe failures
APP="${1:-myapp}"          # first argument, default 'myapp'
LOG="/var/log/${APP}_deploy.log"
log() { echo "$(date '+%F %T') $*" | tee -a "$LOG"; }
log "Deploying $APP"
if [ -d "$APP" ]; then
  log "Directory exists"
else
  mkdir -p "$APP"
fi
for file in *.log; do
  [ -e "$file" ] || continue
  log "Processing $file"
done
if curl -fs http://localhost:8080/health > /dev/null; then
  log "Health check OK"
else
  log "Health check FAILED"; exit 1
fi
```

- **#!/bin/bash** — selects Bash as the interpreter.
- **set -euo pipefail** — makes scripts fail fast instead of silently continuing.
- **${1:-default}** — positional argument with a default value.
- **[-d] / [-f] / [-e]** — directory / regular file / anything exists.
- **function name() { ... }** — reusable block of commands.
- **exit 1** — returns failure status, which CI tools such as Jenkins detect.

Practical point: *$? is the previous command's exit status (0 = success). $1, $2 are arguments, $@ is all arguments, $# is the argument count. Run ShellCheck on scripts to catch bugs early.*

4. Git & GitHub

Definition — Git:

Git is a distributed version-control system that tracks source-code changes. It provides commits, branches, merging and history so teams can work safely on the same project.

Definition — GitHub:

GitHub is a platform for hosting Git repositories and collaborating through pull requests, code review, issues, GitHub Actions and security features. It commonly stores the source code used by CI/CD pipelines.

Setup & Daily Workflow

```
git config --global user.name "Vaibhav Kunghatkar"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git clone <url>
git status -sb
git switch -c feature/login
git add -p                    # stage chunks interactively
git commit -m "Add login form"
git pull --rebase origin main
git push -u origin feature/login
```

Inspect, Undo & Recover

```
git log --oneline --graph --decorate --all
git diff
git diff --staged
git show <commit>
git blame file.txt
git restore file.txt                # discard working changes
git restore --staged file.txt       # unstage
git revert <commit>                 # safe undo (new commit)
git reset --soft HEAD~1             # undo commit, keep changes staged
git reflog                          # find 'lost' commits
git stash push -m "wip"
git stash list
git stash pop
```

Branching, Merging & Collaboration

```
git branch -a
git switch main
git merge feature/login
git rebase main                     # replay your commits on top of main
git cherry-pick <commit>
git tag -a v1.0.0 -m "First release"
git push origin v1.0.0
git remote -v
git fetch --all --prune
git branch -d feature/login
git push origin --delete feature/login
```

- **switch / restore** — modern replacements for the overloaded 'git checkout'.
- **pull --rebase** — keeps history linear by replaying local commits on the latest remote.
- **revert vs reset** — revert is safe on shared branches; reset rewrites history, so use it only on local work.
- **reflog** — your safety net: recover commits after a bad reset or deleted branch.
- **push --force-with-lease** — safer than --force when you must overwrite a rebased branch.

GitHub Essentials

- Pull Request (PR): proposes changes, enables review and runs checks before merge.
- Branch protection / rulesets: require reviews and passing status checks before merging to main.
- GitHub Actions: workflows in .github/workflows/*.yml trigger CI/CD on push, PR or schedule.
- Secrets: store credentials in repository/organization secrets, never in code.
- Authentication: use SSH keys or a personal access token; passwords are no longer accepted for Git over HTTPS.
- Dependabot and secret scanning: automated dependency updates and leaked-credential alerts.

```
# .github/workflows/ci.yml
name: ci
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t myapp:${{ github.sha }} .
```

```
Working Directory --git add--> Staging Area
   --git commit--> Local Repo --git push--> GitHub
```

5. AWS Services - Main Notes

Definition — Amazon Web Services (AWS):

AWS is a cloud platform providing compute, storage, networking, databases, security, containers, monitoring and many other services. DevOps engineers combine these services to build, deploy, secure, scale and monitor applications.

Getting Started with the AWS CLI v2

```
aws --version
aws configure                       # access keys (lab use only)
aws configure sso                   # recommended: IAM Identity Center login
aws sso login --profile dev
aws sts get-caller-identity         # which identity am I using?
aws configure list
export AWS_PROFILE=dev
export AWS_REGION=ap-south-1
```

Practical point: *Prefer short-lived credentials (IAM Identity Center / roles) over long-lived access keys. Never commit keys to Git, and turn on MFA for the root and admin users.*

EC2 (Elastic Compute Cloud)

Definition — EC2:

EC2 provides virtual servers in the AWS cloud. You choose the machine image (AMI), instance type, storage and networking, then run applications or DevOps tools on the server.

- Common uses: web servers, Jenkins servers, Docker hosts, bastion hosts, worker nodes.
- Security Groups: stateful virtual firewalls (allow rules only) attached to instances.
- User Data: a startup script that automates first-boot setup.
- Key pairs / SSM Session Manager: SSH access, or shell access with no open port 22 using Systems Manager.
- IMDSv2: require session tokens for the instance metadata service to reduce SSRF risk.
- Pricing: On-Demand, Savings Plans, Reserved, Spot (cheap but interruptible). Graviton (ARM) instances often give better price/performance.

```
aws ec2 describe-instances --output table \
    --query 'Reservations[].Instances[].[InstanceId,State.Name,PublicIpAddress]'
aws ec2 run-instances --image-id ami-xxxx --instance-type t3.micro \
    --key-name mykey --security-group-ids sg-xxxx --subnet-id subnet-xxxx
aws ec2 stop-instances --instance-ids i-0123456789abcdef0
aws ec2 start-instances --instance-ids i-0123456789abcdef0
aws ec2 terminate-instances --instance-ids i-0123456789abcdef0
aws ec2 describe-security-groups --group-ids sg-xxxx
aws ssm start-session --target i-0123456789abcdef0
```

S3 (Simple Storage Service)

Definition — S3:

S3 is an object-storage service for files, backups, application artifacts, static content and logs. Objects are stored inside buckets.

- Versioning keeps previous object versions; lifecycle rules move or expire objects automatically.

- Storage classes: Standard, Intelligent-Tiering, Standard-IA, Glacier tiers, to balance cost and access speed.
- Block Public Access is on by default for new buckets; keep it on unless hosting a deliberate public site.
- Encryption at rest is applied by default (SSE-S3); use SSE-KMS when you need key control and audit.
- Access is controlled with IAM policies and bucket policies.

```
aws s3 ls
aws s3 mb s3://my-unique-bucket
aws s3 cp file.txt s3://my-unique-bucket/
aws s3 sync ./site s3://my-unique-bucket/ --delete
aws s3 rm s3://my-unique-bucket/old/ --recursive
aws s3 presign s3://my-unique-bucket/file.txt --expires-in 3600
aws s3api put-bucket-versioning --bucket my-unique-bucket \
    --versioning-configuration Status=Enabled
```

IAM (Identity and Access Management)

Definition — IAM:

IAM controls authentication and authorization in AWS. Users, groups, roles and policies define which actions can be performed on which resources.

- Roles: temporary credentials for EC2, Lambda, EKS pods, CI/CD pipelines and cross-account access.
- Least privilege: grant only the permissions actually required; start narrow and widen with evidence.
- Policy evaluation: an explicit Deny always wins; otherwise an Allow is required.
- Root user: protect with MFA and do not use it for daily work.
- IAM Identity Center: central single sign-on for people across accounts.
- IAM Access Analyzer: finds unintended public or cross-account access.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject"],
    "Resource": "arn:aws:s3:::my-unique-bucket/*"
  }]
}
```

```
aws iam list-users
aws iam list-roles --query 'Roles[].RoleName'
aws iam list-attached-role-policies --role-name my-role
aws iam simulate-principal-policy --policy-source-arn <role-arn> \
    --action-names s3:GetObject
```

VPC (Virtual Private Cloud)

Definition — VPC:

A VPC is the networking foundation of AWS: an isolated IP network containing subnets, route tables, gateways and security controls.

- Public subnet: route to an Internet Gateway (IGW). Private subnet: no direct inbound path from the Internet.
- NAT Gateway: gives private subnets outbound Internet access (billed hourly plus data, so watch costs).
- Security Groups (stateful, instance level) vs Network ACLs (stateless, subnet level).
- VPC Endpoints: reach S3, ECR, SSM and other services privately without the Internet.

- Use at least two Availability Zones for high availability.

```
aws ec2 describe-vpcs
aws ec2 describe-subnets --filters Name=vpc-id,Values=vpc-xxxx
aws ec2 describe-route-tables
aws ec2 describe-internet-gateways
aws ec2 describe-nat-gateways
```

6. AWS Services - Compute, Database & Traffic

RDS (Relational Database Service)

Definition — RDS:

RDS is a managed relational database service for engines such as MySQL, PostgreSQL, MariaDB, Oracle and SQL Server (plus Amazon Aurora). AWS manages much of the underlying infrastructure.

- Features: automated backups, snapshots, Multi-AZ failover, read replicas, maintenance windows.
- Keep databases in private subnets; allow access only from the application tier's security group.
- Store credentials in AWS Secrets Manager; enable encryption at rest.
- Aurora is AWS's cloud-native MySQL/PostgreSQL-compatible engine, with Aurora Serverless v2 for variable load.

```
aws rds describe-db-instances \
    --query 'DBInstances[].[DBInstanceIdentifier,DBInstanceStatus,Endpoint.Address]'
aws rds create-db-snapshot --db-instance-identifier mydb \
    --db-snapshot-identifier mydb-snap-01
aws rds reboot-db-instance --db-instance-identifier mydb
```

ELB / ALB (Elastic Load Balancing)

Definition — ELB / ALB:

Elastic Load Balancing distributes incoming traffic across healthy targets. The Application Load Balancer (ALB) works at the HTTP/HTTPS layer; the Network Load Balancer (NLB) works at the TCP/UDP layer.

- Listeners accept traffic; rules route by host, path or header; target groups hold backend targets.
- Health checks prevent traffic going to unhealthy targets.
- Attach ACM certificates for HTTPS and redirect HTTP to HTTPS.
- ALB is the usual choice for web apps and Kubernetes Ingress on EKS; NLB for extreme performance or non-HTTP protocols.

```
aws elbv2 describe-load-balancers
aws elbv2 describe-target-groups
aws elbv2 describe-target-health --target-group-arn <arn>
```

Auto Scaling

- Auto Scaling Groups (ASG) keep the desired number of EC2 instances and scale on metrics.
- Launch templates define AMI, instance type, user data and security groups.
- Combine ASG + ALB + multiple AZs for resilient web tiers.

```
aws autoscaling describe-auto-scaling-groups
aws autoscaling set-desired-capacity --auto-scaling-group-name my-asg --desired-capacity 3
```

Route 53, CloudFront & ECR

- Route 53: DNS service. Alias records point domain names to ALB, CloudFront, S3 websites and more; routing policies include simple, weighted, latency and failover.
- CloudFront: CDN that caches content at edge locations. Origins include S3 and ALB; use Origin Access Control (OAC) to keep S3 private.
- ECR: managed registry for Docker/OCI images with image scanning, lifecycle policies and IAM-based access.

```
aws route53 list-hosted-zones
aws cloudfront create-invalidation --distribution-id EXXXX --paths "/*"
aws ecr create-repository --repository-name myapp
aws ecr get-login-password --region ap-south-1 | \
  docker login --username AWS --password-stdin <acct-id>.dkr.ecr.ap-south-1.amazonaws.com
docker tag myapp:1.0 <acct-id>.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0
docker push <acct-id>.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0
```

EKS (Elastic Kubernetes Service)

EKS is AWS's managed Kubernetes service. AWS runs and patches the control plane; your workloads run on managed node groups, Fargate, Karpenter-managed nodes or EKS Auto Mode. Details are in Section 12.

7. AWS Services - Monitoring & Serverless

CloudWatch (Monitoring & Observability)

Definition — CloudWatch:

CloudWatch collects AWS and application metrics and logs, and supports alarms, dashboards and event-driven reactions.

- Metrics: CPU, network, request counts, custom metrics. Logs: log groups and streams; query with Logs Insights.
- Alarms: notify through SNS or trigger Auto Scaling when thresholds are crossed.
- Container Insights: metrics for ECS and EKS. CloudWatch agent: ships memory, disk and logs from EC2.
- Related services: CloudTrail (who did what: API audit), AWS Config (resource compliance), X-Ray (tracing).

```
aws logs describe-log-groups
aws logs tail /aws/lambda/my-func --follow
aws cloudwatch list-metrics --namespace AWS/EC2
aws cloudwatch describe-alarms --state-value ALARM
aws cloudtrail lookup-events --max-results 5
```

Lambda (Serverless Compute)

Definition — Lambda:

Lambda runs code in response to events without the customer managing servers. You pay for execution time and requests.

- Triggers: S3 events, API Gateway, EventBridge schedules, SQS, SNS, DynamoDB streams and more.
- Use cases: automation (stop dev instances at night), image processing, lightweight APIs, glue between AWS services.
- Key settings: runtime, memory, timeout, execution role (IAM), environment variables, concurrency.

```
aws lambda list-functions --query 'Functions[].FunctionName'
aws lambda invoke --function-name my-func --payload '{"key":"value"}' \
    --cli-binary-format raw-in-base64-out out.json
aws lambda update-function-code --function-name my-func --zip-file fileb://function.zip
```

Other Useful Services

- **SNS / SQS** — notifications (pub/sub) and message queues for decoupling services.
- **EventBridge** — event bus and scheduler for automation.
- **Secrets Manager / SSM Parameter Store** — store credentials and configuration securely.
- **DynamoDB** — serverless NoSQL key-value database.
- **ECS / Fargate** — AWS-native container orchestration, simpler than Kubernetes.
- **AWS Budgets / Cost Explorer** — track spend and set alerts; essential for lab accounts.

3-Tier Architecture

```
Internet
   |
Route 53  ->  (CloudFront optional)
   |
ALB  (public subnets, 2+ AZs)
   |
Web / App Tier  (private subnets, Auto Scaling)
   |
RDS  (private DB subnets, Multi-AZ)
S3 -> files / backups     CloudWatch -> logs / metrics     IAM -> permissions
```

A 3-tier architecture separates web, application and database layers. It gives clear security boundaries (security groups chain from ALB to app to DB) and lets each layer scale independently.

8. Docker

> **Definition — Docker:**
>
> Docker is a container platform used to package an application and its dependencies into a portable image. That image runs consistently across development, CI and cloud environments.

Why DevOps uses Docker: containers make deployments repeatable, reduce differences between environments, start in seconds and fit naturally into CI/CD pipelines and registries such as ECR.

Essential Commands

```
docker version
docker pull nginx:1.27
docker images
docker build -t myapp:1.0 .
docker run -d --name myapp -p 8080:80 myapp:1.0
docker run --rm -it ubuntu:24.04 bash
docker ps
docker ps -a
docker logs -f --tail 100 myapp
docker exec -it myapp sh
docker inspect myapp
docker stats
docker stop myapp && docker rm myapp
docker rmi myapp:1.0
```

- **-d** — detached/background mode.
- **-p 8080:80** — maps host port 8080 to container port 80.
- **--rm** — removes the container automatically when it exits.
- **-it** — interactive terminal, used for shells.
- **logs -f --tail** — follow the last N lines of output.
- **inspect** — full JSON detail: IP, mounts, env, health status.
- **stats** — live CPU/memory per container.

Volumes, Networks & Environment

```
docker run -d --name db -e POSTGRES_PASSWORD=secret \
    -v pgdata:/var/lib/postgresql/data postgres:16
docker volume ls
docker network create appnet
docker run -d --network appnet --name web -p 80:80 nginx
docker run --env-file .env -p 3000:3000 myapp:1.0
```

Cleanup

```
docker system df
docker container prune
docker image prune -a
docker volume prune
docker system prune -a --volumes   # removes unused data; be careful
```

Dockerfile (good practice example)

```
# syntax=docker/dockerfile:1
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
FROM nginx:1.27-alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
HEALTHCHECK CMD wget -qO- http://localhost/ || exit 1
```

- **FROM** — selects the base image; pin a version tag rather than relying on 'latest'.
- **WORKDIR / COPY / RUN** — set directory, copy files, execute build steps.
- **Multi-stage build** — build in one stage, ship only the output in a small final image.
- **EXPOSE** — documents the container port (does not publish it).
- **CMD vs ENTRYPOINT** — default command vs fixed executable; CMD is easy to override.
- **.dockerignore** — excludes .git, node_modules, secrets from the build context.

Practical point: *Run containers as a non-root user (USER app), keep images small, order Dockerfile layers so rarely-changing steps come first (better caching), and never bake secrets into images.*

Docker Compose v2 (multi-container apps)

```
# compose.yaml
services:
  web:
    build: .
    ports: ["8080:80"]
    depends_on: [db]
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    volumes: [pgdata:/var/lib/postgresql/data]
volumes:
  pgdata:
```

```
docker compose up -d --build
docker compose ps
docker compose logs -f web
docker compose exec web sh
docker compose down            # add -v to remove volumes too
```

Buildx, Registries & Scanning

```
docker buildx build --platform linux/amd64,linux/arm64 \
    -t myrepo/myapp:1.0 --push .
docker login
docker tag myapp:1.0 myrepo/myapp:1.0
docker push myrepo/myapp:1.0
docker scout quickview myapp:1.0     # vulnerability summary (Docker Scout)
```

```
Code -> Docker Build -> Image -> Trivy Scan -> Amazon ECR -> EKS / ECS
```

9. Jenkins & CI/CD

> **Definition — Jenkins:**
>
> Jenkins is an open-source automation server commonly used to create CI/CD pipelines. It fetches source code, runs tests and analysis, builds images and triggers deployments.

Why DevOps uses Jenkins: it connects many development and operations tools (Git, Docker, SonarQube, Trivy, AWS, Kubernetes) into one repeatable workflow. A Git push can trigger automated build, test, scan and deploy stages.

Install & Run (Ubuntu example)

```
sudo apt update
sudo apt install -y fontconfig openjdk-21-jre   # Jenkins needs JDK 17 or 21
# add the official Jenkins LTS apt repository, then:
sudo apt install -y jenkins
sudo systemctl enable --now jenkins
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
# UI: http://<server-ip>:8080  (open 8080 in the Security Group)
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

Practical point: *Use the LTS release line for stability, keep plugins updated, enable authentication/authorization, and do not expose Jenkins to the whole Internet.*

Typical CI/CD Flow

```
Checkout -> Build -> Test -> SonarQube -> Docker Build
   -> Trivy Scan -> Push to ECR -> Deploy to EKS
```

Declarative Jenkinsfile

```
pipeline {
  agent any
  environment {
    REGION = 'ap-south-1'
    REPO   = '123456789012.dkr.ecr.ap-south-1.amazonaws.com/myapp'
    TAG    = "${env.BUILD_NUMBER}"
  }
  options { timestamps(); timeout(time: 30, unit: 'MINUTES') }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build & Test') { steps { sh 'npm ci && npm test' } }
    stage('Docker Build') { steps { sh 'docker build -t $REPO:$TAG .' } }
    stage('Trivy Scan') {
      steps { sh 'trivy image --severity HIGH,CRITICAL --exit-code 1 $REPO:$TAG' }
    }
    stage('Push to ECR') {
      steps {
        sh '''
          aws ecr get-login-password --region $REGION | \
            docker login --username AWS --password-stdin ${REPO%%/*}
          docker push $REPO:$TAG
        '''
      }
    }
    stage('Deploy') {
      steps { sh 'kubectl set image deployment/myapp myapp=$REPO:$TAG' }
    }
  }
  post {
    success { echo 'Pipeline succeeded' }
    failure { echo 'Pipeline failed' }
    always  { cleanWs() }
  }
}
```

- **pipeline / agent / stages / steps** — structure of a declarative pipeline.
- **environment** — variables available to all stages.
- **options** — pipeline settings such as timeout and timestamps.
- **post** — actions after the run: success, failure, always.
- **sh** — runs a Linux shell command.
- **credentials()** — injects secrets stored in Jenkins Credentials (never hard-code secrets).
- **when { branch 'main' }** — runs a stage only for certain conditions.

Key Concepts

- Jenkinsfile in the repo = Pipeline as Code (versioned and reviewed like application code).
- Triggers: GitHub webhook (preferred), SCM polling or a schedule (cron syntax).
- Agents/nodes run builds; controller schedules them. Use Docker or Kubernetes agents for clean builds.
- Credentials plugin stores secrets; Blue Ocean/Stage View visualise runs.
- Alternative CI tools: GitHub Actions, GitLab CI, AWS CodePipeline.

Useful Jenkins Troubleshooting

```
sudo systemctl status jenkins
sudo journalctl -u jenkins -f
sudo tail -f /var/log/jenkins/jenkins.log
sudo -u jenkins docker ps          # does the jenkins user have Docker access?
df -h /var/lib/jenkins             # full disk is a common cause of failures
```

10. Terraform

> **Definition — Terraform:**
>
> Terraform is an Infrastructure as Code (IaC) tool that describes cloud infrastructure in configuration files (HCL). You declare the desired state and Terraform calculates the changes required to reach it.

Why DevOps uses Terraform: infrastructure becomes version-controlled, reviewable, reusable and reproducible. OpenTofu is a community-maintained, Terraform-compatible fork that uses nearly identical syntax and workflow.

Core Workflow

```
terraform version
terraform init                    # download providers/modules, set up backend
terraform fmt -recursive
terraform validate
terraform plan -out=tfplan        # preview changes, save the plan
terraform apply tfplan            # apply exactly what was reviewed
terraform destroy
terraform output
terraform show
```

- **init** — initialises providers, modules and backend.
- **fmt** — formats code to canonical style.
- **validate** — checks configuration syntax and consistency.
- **plan -out** — previews and saves the change set for review.
- **apply tfplan** — applies the saved plan, so no surprises.
- **destroy** — removes everything managed by the configuration; use with care.

State & Utility Commands

```
terraform state list
terraform state show aws_s3_bucket.demo
terraform state mv <old> <new>
terraform state rm <resource>
terraform import aws_s3_bucket.demo my-existing-bucket
terraform workspace list
terraform workspace new staging
terraform apply -var="env=prod" -var-file=prod.tfvars
terraform apply -target=aws_instance.web      # last resort, not routine
terraform force-unlock <LOCK_ID>
TF_LOG=DEBUG terraform plan                   # verbose logging
```

Example Configuration

```
terraform {
  required_version = ">= 1.9"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 6.0" }
  }
  backend "s3" {
    bucket       = "my-tf-state-bucket"
    key          = "dev/terraform.tfstate"
    region       = "ap-south-1"
    encrypt      = true
    use_lockfile = true      # S3 native state locking (newer Terraform releases)
  }
}
provider "aws" {
  region = var.region
}
variable "region" {
  type    = string
  default = "ap-south-1"
}
resource "aws_s3_bucket" "demo" {
  bucket = "my-unique-demo-bucket"
  tags   = { Env = "dev" }
}
output "bucket_arn" {
  value = aws_s3_bucket.demo.arn
}
```

- **provider** — plugin that talks to a platform (AWS, Azure, Kubernetes...).
- **resource** — an infrastructure object Terraform manages.
- **data** — reads existing information without managing it.
- **variable / output / locals** — inputs, exposed results and computed values.
- **module** — reusable group of resources; use the Terraform Registry or your own.
- **backend + locking** — store state remotely (S3) with locking so teams do not corrupt state.
- **lifecycle** — controls create_before_destroy, prevent_destroy, ignore_changes.

Practical point: *The state file can contain sensitive values: never commit it to Git, and always use a remote, encrypted backend for team work. Always read the plan before apply, and watch for 'destroy' and 'replace' lines.*

11. Kubernetes

> **Definition — Kubernetes:**
>
> Kubernetes (K8s) is an open-source container orchestration platform. It manages containerised workloads and provides scheduling, service discovery, scaling, self-healing and rolling updates.

Why DevOps uses Kubernetes: it keeps the desired number of application instances running and offers a standard way to deploy, update and connect containers across environments.

Cluster & Context

```
kubectl version
kubectl cluster-info
kubectl config get-contexts
kubectl config use-context <name>
kubectl config set-context --current --namespace=dev
kubectl get nodes -o wide
kubectl top nodes             # needs metrics-server
```

Inspect Resources

```
kubectl get pods
kubectl get pods -A
kubectl get pods -n dev -o wide
kubectl get pods --show-labels
kubectl get svc,deploy,rs,ingress
kubectl get all -n dev
kubectl get pod <pod> -o yaml
kubectl describe pod <pod>
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl top pods
kubectl api-resources
kubectl explain deployment.spec.strategy
```

Logs & Debugging

```
kubectl logs <pod>
kubectl logs -f <pod> -c <container>
kubectl logs <pod> --previous          # logs from the crashed container
kubectl exec -it <pod> -- sh
kubectl port-forward svc/myapp 8080:80
kubectl debug -it <pod> --image=busybox --target=<container>
kubectl run tmp --rm -it --image=busybox -- sh
```

Create, Update & Roll Back

```
kubectl apply -f deployment.yaml
kubectl apply -f k8s/                  # whole folder
kubectl diff -f deployment.yaml        # preview what would change
kubectl create deployment web --image=nginx --replicas=2
kubectl expose deployment web --port=80 --type=ClusterIP
kubectl scale deployment web --replicas=5
kubectl set image deployment/web nginx=nginx:1.27
kubectl rollout status deployment/web
kubectl rollout history deployment/web
kubectl rollout undo deployment/web
kubectl rollout restart deployment/web
kubectl delete -f deployment.yaml
kubectl delete pod <pod>               # Deployment will recreate it
kubectl create deployment web --image=nginx --dry-run=client -o yaml > web.yaml
```

Practical point: *--dry-run=client -o yaml is the quickest way to generate a YAML skeleton instead of writing from memory.*

Sample Deployment + Service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
  selector:
    matchLabels: { app: myapp }
  template:
    metadata:
      labels: { app: myapp }
    spec:
      containers:
        - name: myapp
          image: myrepo/myapp:1.0
          ports: [{ containerPort: 80 }]
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits:   { cpu: "500m", memory: "256Mi" }
          readinessProbe:
            httpGet: { path: /health, port: 80 }
          livenessProbe:
            httpGet: { path: /health, port: 80 }
---
apiVersion: v1
kind: Service
metadata:
  name: myapp
spec:
  selector: { app: myapp }
  ports: [{ port: 80, targetPort: 80 }]
  type: ClusterIP
```

Core Objects

- **Pod** — smallest deployable unit; one or more containers sharing network and storage.

- **Deployment** — manages desired Pods and rolling updates.
- **ReplicaSet** — maintains the desired number of replicas.
- **Service** — stable network endpoint: ClusterIP, NodePort, LoadBalancer.
- **Ingress** — HTTP/HTTPS routing into the cluster (needs an ingress controller).
- **ConfigMap / Secret** — non-secret configuration / sensitive data (base64-encoded, so also use RBAC and encryption).
- **Namespace** — logical partition of a cluster.
- **StatefulSet / DaemonSet / Job / CronJob** — stateful apps, one pod per node, one-off tasks, scheduled tasks.
- **PV / PVC / StorageClass** — persistent storage and how it is requested.
- **HPA** — Horizontal Pod Autoscaler: scales replicas on CPU, memory or custom metrics.
- **RBAC / ServiceAccount** — permissions for users and pods.
- **NetworkPolicy** — controls which pods may talk to which.

```
Deployment -> ReplicaSet -> Pods -> Containers
```

Helm (package manager for Kubernetes)

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo nginx
helm install web bitnami/nginx -n dev --create-namespace
helm list -A
helm upgrade web bitnami/nginx -f values.yaml
helm rollback web 1
helm uninstall web -n dev
```

Common Pod Problems

- **Pending** — no node can fit it: check resources, taints, PVCs (kubectl describe pod).
- **ImagePullBackOff / ErrImagePull** — wrong image name/tag or missing registry credentials.
- **CrashLoopBackOff** — container keeps crashing: kubectl logs --previous and check config/probes.
- **OOMKilled** — memory limit exceeded: raise limit or fix the leak.
- **Service has no endpoints** — selector labels do not match pod labels (kubectl get endpoints).

12. AWS EKS

> **Definition — Amazon EKS:**
>
> Amazon EKS is AWS's managed Kubernetes service. AWS manages the Kubernetes control plane; workloads run on managed node groups, Fargate, Karpenter-provisioned EC2 nodes, or EKS Auto Mode, where AWS also manages compute, networking and storage add-ons.

Why DevOps uses EKS: it combines Kubernetes orchestration with AWS networking (VPC), IAM, load balancing and container registries such as ECR.

Architecture

```
Developer -> GitHub -> CI/CD -> ECR
                                 |
                               EKS  (control plane managed by AWS)
                                 |
                          Service / Ingress
                                 |
                     AWS Load Balancer (ALB / NLB)
                                 |
                               Users
```

Create a Cluster & Connect

```
# eksctl (quickest for learning)
eksctl create cluster --name demo --region ap-south-1 \
    --nodegroup-name ng1 --node-type t3.medium --nodes 2
eksctl get cluster
eksctl delete cluster --name demo       # avoid surprise bills
# connect kubectl to an existing cluster
aws eks list-clusters
aws eks update-kubeconfig --name demo --region ap-south-1
kubectl get nodes
```

Key Components

- **Managed Node Group** — AWS-managed EC2 worker capacity with automated updates.
- **Fargate profile** — serverless pods with no nodes to manage.
- **Karpenter / Cluster Autoscaler** — adds and removes nodes based on pending pods.
- **EKS add-ons** — managed VPC CNI, CoreDNS, kube-proxy, EBS CSI driver.
- **AWS Load Balancer Controller** — creates ALB for Ingress and NLB for Services of type LoadBalancer.
- **EKS Pod Identity / IRSA** — give pods their own least-privilege IAM role instead of sharing the node role.
- **Access entries** — EKS-native way to map IAM principals to Kubernetes permissions (replacing the aws-auth ConfigMap).
- **ECR** — stores container images; nodes pull with their IAM role.

Deploy an App with an ALB Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
spec:
  ingressClassName: alb
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: myapp, port: { number: 80 } }
```

```
kubectl apply -f ingress.yaml
kubectl get ingress myapp          # ADDRESS column shows the ALB DNS name
```

EKS Troubleshooting

- kubectl get nodes: are nodes Ready? If not, check the node group, subnets and IAM node role.
- kubectl get pods -A: look for Pending, CrashLoopBackOff or ImagePullBackOff.
- kubectl describe pod: read the Events section first.
- kubectl logs (and --previous): inspect the application output.
- kubectl get svc,endpoints,ingress: confirm selectors match and the load balancer was created.
- Check IAM (node role, Pod Identity/IRSA, access entries), security groups, subnet tags and available IPs in the subnets.
- kubectl -n kube-system get pods: CoreDNS, CNI and the Load Balancer Controller must be healthy.

```
kubectl -n kube-system logs deploy/aws-load-balancer-controller
aws eks describe-cluster --name demo --query cluster.status
aws eks describe-nodegroup --cluster-name demo --nodegroup-name ng1
```

13. DevSecOps & Monitoring

> **Definition — DevSecOps:**
>
> A software delivery approach that places security checks throughout the development and CI/CD lifecycle ('shift left') instead of treating security as a final step.

SonarQube

Code quality and analysis platform that finds bugs, code smells, duplications and security hotspots. Quality Gates can fail a pipeline when standards are not met.

```
sonar-scanner -Dsonar.projectKey=myapp -Dsonar.sources=. \
    -Dsonar.host.url=http://sonar:9000 -Dsonar.token=$SONAR_TOKEN
```

Trivy

Open-source vulnerability scanner for container images, filesystems, Git repos, IaC (Terraform/Kubernetes manifests) and secrets.

```
trivy image myapp:1.0
trivy image --severity HIGH,CRITICAL --exit-code 1 myapp:1.0
trivy fs .
trivy config ./terraform            # scan IaC for misconfigurations
trivy k8s --report summary cluster
```

More Security Practices

- Secrets: AWS Secrets Manager / SSM Parameter Store / Jenkins credentials; never plain text in Git.
- Least privilege IAM and short-lived credentials; MFA everywhere.
- Dependency scanning (Dependabot, npm audit, pip-audit) and image signing (Cosign).
- SAST (code), DAST (running app) and SCA (dependencies) checks in the pipeline.
- AWS services: GuardDuty (threat detection), Security Hub, Inspector (vulnerabilities), CloudTrail (audit), AWS Config.

Monitoring Stack

- **CloudWatch** — AWS-native metrics, logs, alarms and dashboards.
- **Prometheus** — metrics collection and alerting, the standard for Kubernetes.
- **Grafana** — dashboards on top of Prometheus, CloudWatch and others.
- **Alertmanager / SNS** — route alerts to email, Slack or on-call tools.
- **Loki / ELK / OpenSearch** — centralised log aggregation.

```
helm repo add prometheus-community \
    https://prometheus-community.github.io/helm-charts
helm install monitoring prometheus-community/kube-prometheus-stack \
    -n monitoring --create-namespace
kubectl -n monitoring port-forward svc/monitoring-grafana 3000:80
```

14. Complete DevOps Pipeline

```
Developer
   |
GitHub (PR + review + branch protection)
   |
Jenkins / GitHub Actions
   |
Build + Unit Tests
   |
SonarQube (quality gate)
   |
Docker Build
   |
Trivy Scan (fail on HIGH/CRITICAL)
   |
Amazon ECR
   |
Kubernetes / EKS  (Terraform provisions the infrastructure)
   |
Ingress / ALB
   |
Application
   |
CloudWatch / Prometheus + Grafana
```

This combines source control, automation, testing, code analysis, containers, security scanning, image storage, Kubernetes deployment, infrastructure as code and monitoring.

Where each tool fits

- **Linux + shell** — the base: servers, scripts and automation.
- **Git + GitHub** — source of truth and collaboration; triggers the pipeline.
- **Terraform** — creates VPC, EKS, ECR, RDS and other AWS infrastructure.
- **Jenkins** — orchestrates build, test, scan and deploy stages.
- **Docker + ECR** — packages and stores the application image.
- **Kubernetes / EKS** — runs, scales and heals the application.
- **CloudWatch / Prometheus** — tells you when something breaks.

GitOps note

A popular modern pattern: CI builds and pushes the image, then updates a Git repository holding Kubernetes manifests; a tool such as Argo CD or Flux running in the cluster pulls and applies the change, so Git is the single source of truth for deployments.

15. Troubleshooting Checklist

- Read the exact error message first, and note the time it happened.
- Linux service: systemctl status <svc>, then journalctl -u <svc> -xe.
- Resources: top, free -h, df -h (also df -i for inodes), uptime.
- Network: ss -tulpn (is it listening?), ping, curl -v, nc -zv, dig.
- Permissions: ls -l, id, namei -l <path>, check SELinux/AppArmor if all else fails.
- Git: git status, git log --oneline, git reflog; use git diff before committing.
- Docker: docker ps -a, docker logs, docker inspect, docker stats.
- Kubernetes: get pods, describe, logs (--previous), get events, get endpoints.
- Terraform: validate, plan before apply, read the plan; use TF_LOG=DEBUG, check state locks.
- Jenkins: console output of the failed stage, agent disk space, credentials, Docker permission.
- AWS: aws sts get-caller-identity (right account/role?), IAM policy, security groups, NACLs, route tables, subnets, service quotas, CloudTrail events.
- Change one thing at a time, and write down what you tried.

Quick Symptom Table

Symptom	First checks
Website not reachable	service running? port listening? security group/NACL? DNS? ALB target health
Permission denied (SSH)	key file chmod 400, correct user (ubuntu/ec2-user), correct key pair, SG port 22
AccessDenied (AWS)	which identity (sts), IAM policy, resource policy, SCP, region
Pod CrashLoopBackOff	kubectl logs --previous, env/config, probes, resource limits
Disk full	df -h, du -sh /var/*, docker system df, old logs, journalctl --vacuum-size=200M
Terraform state lock	check who is running apply; terraform force-unlock only if sure
Jenkins build fails on docker	jenkins user in docker group, Docker daemon running, restart Jenkins

16. Study Method

```
Learn -> Run -> Break -> Troubleshoot -> Explain
```

Understand the definition, run the command in a lab, intentionally cause a small failure, troubleshoot it using evidence (logs, status, events), and then explain the concept in simple English.

Suggested Learning Order

- 1. Linux and shell scripting: be comfortable on a terminal first.
- 2. Git and GitHub: commits, branches, pull requests.
- 3. AWS core: IAM, EC2, S3, VPC, then RDS, ALB, Route 53, CloudWatch.
- 4. Docker: build, run and push images; then Docker Compose.
- 5. Terraform: provision the AWS pieces you built by hand.
- 6. Jenkins (or GitHub Actions): automate build, test and image push.
- 7. Kubernetes basics locally (kind or minikube), then EKS.
- 8. DevSecOps and monitoring: Trivy, SonarQube, Prometheus/Grafana, CloudWatch.

Practice Project Ideas

- Deploy a static site on S3 + CloudFront with Terraform.
- Run a Dockerised app on EC2 behind an ALB.
- Build a Jenkins pipeline: GitHub -> test -> Docker build -> Trivy -> ECR.
- Deploy the image to EKS with an ALB Ingress and watch it in CloudWatch/Grafana.

Practical point: *Always set an AWS Budget alert and delete lab resources (terraform destroy, eksctl delete cluster) when you finish. NAT Gateways, load balancers and EKS clusters keep billing while idle.*

17. Final Revision Notes

- Linux: commands, permissions, processes, systemd, networking, logs.
- Shell: variables, conditions, loops, exit codes, set -euo pipefail.
- Git/GitHub: add, commit, branch, merge/rebase, pull requests, Actions, undo with restore/revert/reflog.
- AWS: compute (EC2), storage (S3), IAM (least privilege, roles), VPC, databases (RDS), ALB, Route 53, CloudWatch, Lambda.
- Docker: image vs container, Dockerfile, multi-stage builds, volumes, networks, Compose, registries.
- Jenkins: Jenkinsfile stages, credentials, webhooks, post actions, CI/CD automation.
- Terraform: init, plan, apply, destroy, state, remote backend with locking, modules.
- Kubernetes: Pods, Deployments, Services, Ingress, ConfigMaps/Secrets, probes, rollouts, HPA, Helm.
- EKS: control plane vs nodes, ECR, ALB controller, IAM (Pod Identity/IRSA), access entries, autoscaling.
- DevSecOps: scan code and images in the pipeline; monitor everything.

Notes created by Vaibhav Kunghatkar